Beyond Code Generation: Orchestrating GitHub Copilot Agents to Build a Complete ASP.NET Application

Jeffrey Bramlett

October 2026

Engineering Case Study | AdventureWorks2022 | GitHub Copilot Multi-Agent Development

Abstract

This paper reports the results of a practical experiment in agentic software engineering: starting with the Microsoft AdventureWorks2022 relational database as the principal source artifact, could a coordinated team of specialized GitHub Copilot agents infer a useful application, define its architecture and product behavior, implement the frontend and backend, establish safe database access, test the result, and drive the work to a fully operational local website?

The experiment evolved from a small set of specialist agents into an eight-agent software organization supported by a separate coordinating Orchestrator. The specialist roles were API Engineer, Business Expert, Database Engineer, Deployment Engineer, Finance Controller, Frontend Engineer, Product Designer, and Project Architect. The Orchestrator managed task progression, agent handoffs, validation, backlog state, an audit history, and persistent operational artifacts. Human participation was retained for direction, authority, policy, risk acceptance, and decisions that the agents correctly identified as outside their delegated authority.

The completed project contains 44 formally tracked backlog items, 38 of which were completed; 241 meaningful audit records; approximately 9,719 lines of C# and 3,279 lines of JavaScript/JSX; 287 passing API tests in the final recorded validation; 53 passing client tests; and a verified end-to-end order-entry workflow that persisted and subsequently retrieved a real order in an isolated AdventureWorks2022 test clone. The final browser-driven validation created sales order SO75128 and verified the persisted order header and line against the application confirmation.

The central result is not that AI can generate code. Modern coding assistants already demonstrate that capability. The more consequential result is that specialized agents became substantially more useful when placed inside a disciplined engineering system with explicit roles, persistent organizational memory, bounded authority, verification, and an orchestration layer. In this experiment, the human engineer did not disappear. The human role shifted upward in the decision hierarchy, while implementation, analysis, coordination, and much of the verification were delegated to the agent organization.

Keywords: agentic software engineering, GitHub Copilot, ASP.NET Core, React, AdventureWorks2022, AI agents, software architecture, orchestration, human-in-the-loop engineering

8	44	287	53
specialist agents	backlog items	API tests passed	client tests passed

1. Introduction



AI-assisted programming is often discussed in terms of code generation: how quickly a model can create a function, scaffold a component, repair a failing test, or produce a block of SQL. Those capabilities are useful, but they do not represent the full problem of software engineering. A functioning application is the result of thousands of interdependent decisions involving product intent, architecture, data semantics, security, testing, deployment, operational procedures, and authority over consequential actions.

This experiment was designed to probe that larger problem.

The starting point was deliberately constrained. Instead of providing a conventional product requirements document, an existing application, UI design, or detailed backlog, I started with the AdventureWorks2022 database and asked an agent-based development process to work forward from the database to a usable application. AdventureWorks is useful for this purpose because it is neither trivial nor opaque. Its schema contains recognizable business concepts - products, customers, offers, addresses, shipping methods, orders, inventory, and financial fields - but those structures do not, by themselves, specify what application should be built around them.

The original experiment used five specialized agents: Product Designer, Project Architect, Frontend Engineer, API Engineer, and Database Engineer. As the project progressed, additional needs became visible. Business policy could not safely be inferred from schema. Financial rules required a distinct authority. Deployment and runtime behavior required a dedicated operational role. The project therefore evolved to eight specialist agents and a separate Orchestrator responsible for coordinating their work.

That evolution became one of the most important findings. Specialization improved expertise, but specialization alone did not create a team. The project began to behave like an engineering organization only when agent responsibilities, decision boundaries, handoffs, persistent records, validation, and task progression were explicitly coordinated.

The completed repository therefore provides two outputs. The first is the application itself: a React 19 frontend, ASP.NET Core 10 API, SQL Server integration, automated tests, local deployment workflow, and complete order-entry demonstration. The second is the engineering record: agent definitions, backlog, audit history, design documents, API contracts, data notes, runbooks, scripts, tests, and decisions that show how the system was produced.

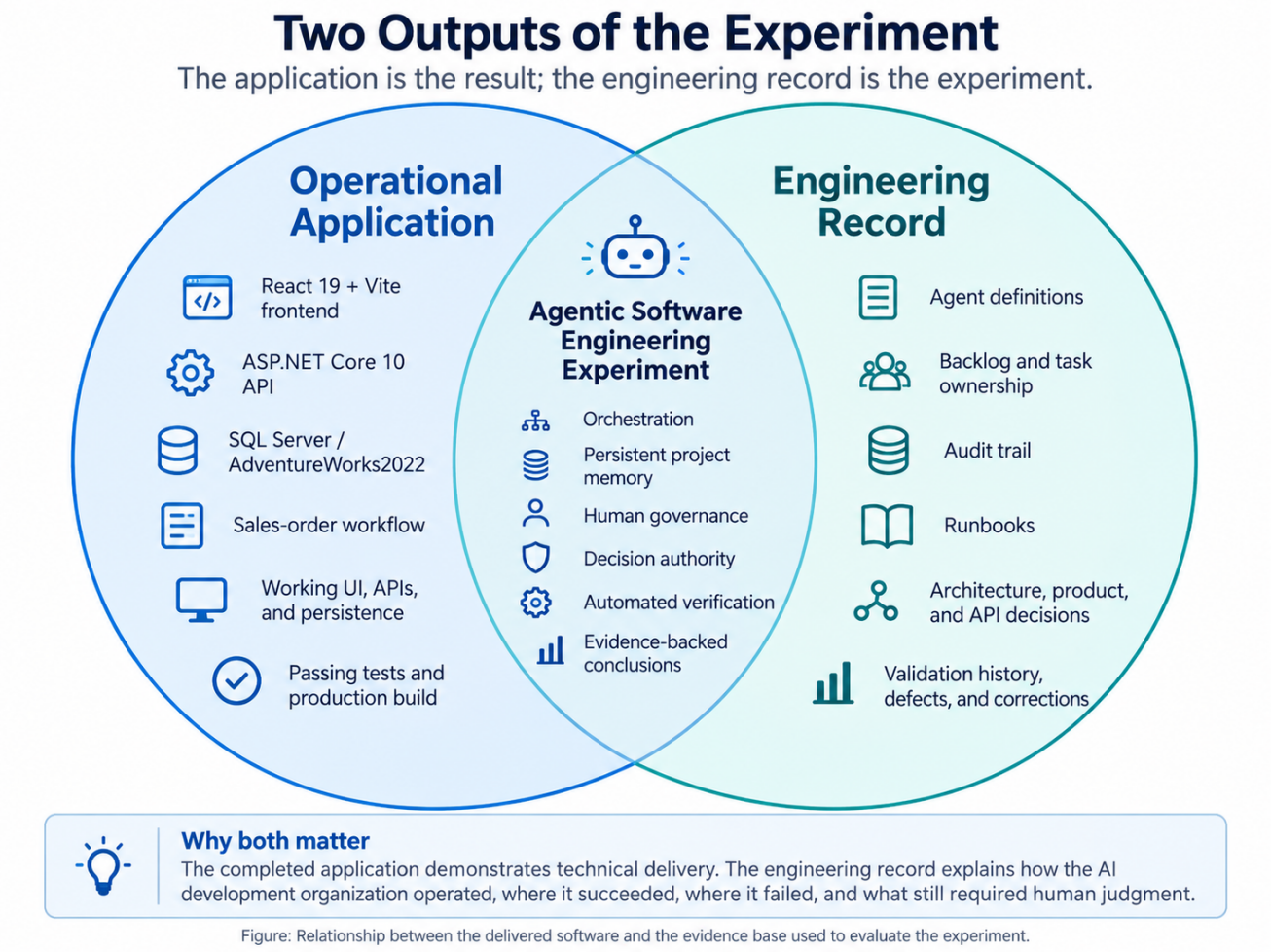


Figure: Relationship between the delivered software and the evidence base used to evaluate the experiment.

This paper analyzes both.

2. Research Questions

The project was organized around a primary question:

> *Can a coordinated team of specialized GitHub Copilot agents, starting principally from an existing enterprise relational database, produce a coherent and operational ASP.NET web application when supported by persistent engineering artifacts and human governance?*

That question decomposes into several more specific questions:

1. Can agents infer a plausible application and user workflow from database structure rather than from a conventional requirements document?
2. Can specialized agents divide work in a manner that produces compatible frontend, API, database, architecture, finance, and deployment decisions?

3. Does an orchestration layer materially improve coordination, continuity, and progression of multi-agent work?
4. Can agents distinguish verified database facts from unresolved business policy rather than silently inventing requirements?
5. Can the agent organization identify, escalate, and correct its own implementation and security problems?
6. What classes of decisions still require a human engineer?
7. Can the resulting application be validated through an actual end-to-end business transaction rather than by compilation or isolated unit tests alone?

The experiment was not intended to establish that the same results would occur for every software project, team, model, or database. It is a documented case study of one bounded application-development effort.

3. Experimental Constraint: Begin with the Database

The central constraint was intentionally unusual: AdventureWorks2022 was treated as the principal starting artifact.

A relational database contains a large amount of implicit information. Table and column names suggest concepts. Foreign keys expose relationships. Check constraints and data types constrain valid values. Triggers reveal side effects. Sample data can reveal patterns of actual use. These artifacts can help reconstruct a domain.

But a database is not a product specification.

For example, the existence of `Sales.SalesOrderHeader` and `Sales.SalesOrderDetail` makes it reasonable to infer that orders exist, but it does not establish which users should create them, what workflow should precede submission, what tax policy should apply, whether prices should be considered authoritative, what constitutes eligible inventory, which authentication method should be used, or what application state should be shown after a failure.

The experiment therefore required the agents to make a disciplined distinction between three categories:

- **Verified facts**: statements supported by repository or database evidence.
- **Provisional recommendations**: sensible behaviors inferred from context but not yet approved as policy.
- **Authorized decisions**: product, finance, security, operational, or data-write decisions explicitly accepted by the responsible authority.

This separation became increasingly important as the application moved from read-only catalog behavior toward order submission.

4. The Agent Organization

4.1 Eight specialist agents

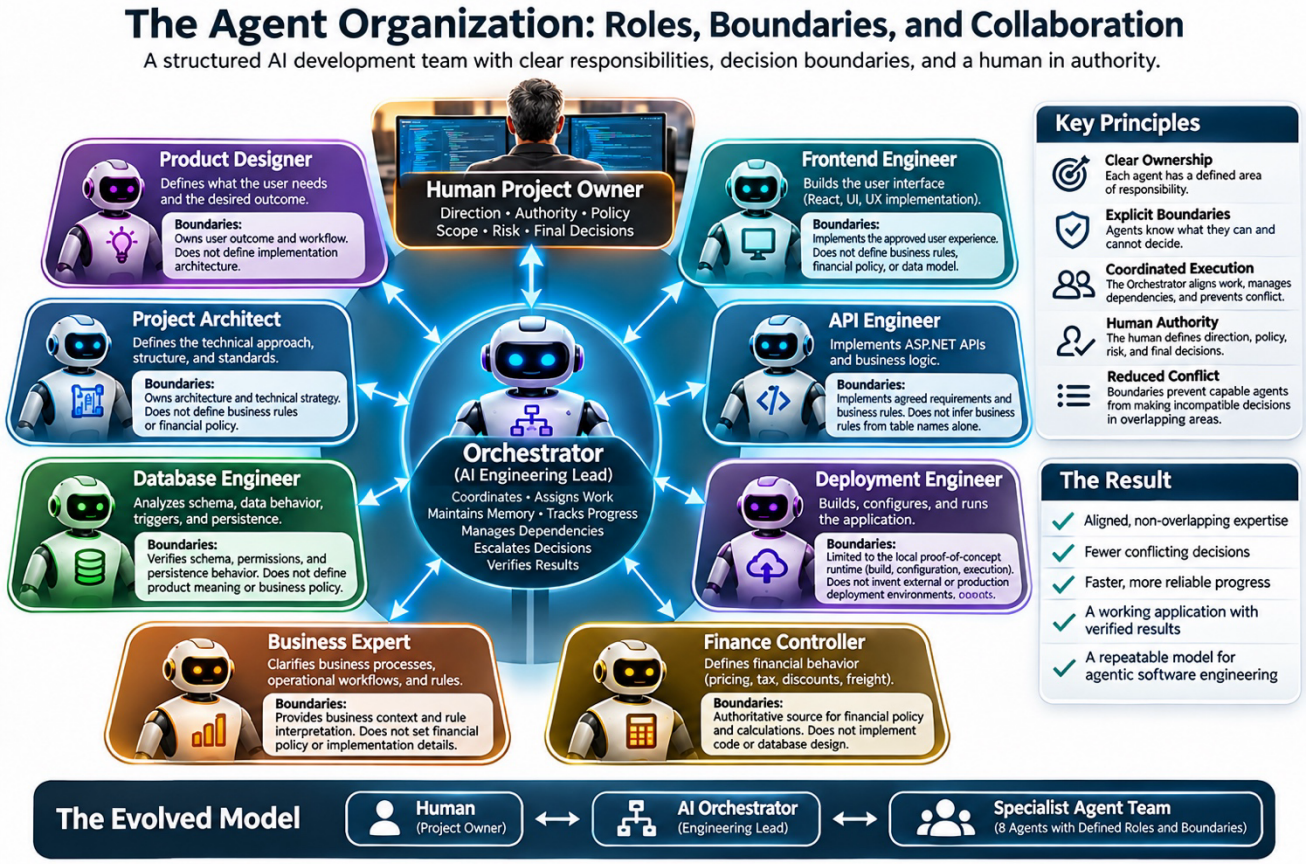
The final project contains eight specialist agent definitions under `.github/agents/`.

Agent	Primary responsibility

Agent	Primary responsibility
Product Designer	Product scope, user outcomes, prioritization, acceptance criteria, and coherent feature sequencing
Project Architect	System boundaries, architecture, contracts, security model, testing strategy, and implementation slices
Frontend Engineer	React UI, interaction behavior, accessibility, client-side validation, and frontend tests
API Engineer	ASP.NET API behavior, server-side validation, business functions, persistence integration, and API tests
Database Engineer	Schema verification, SQL behavior, permissions, least-privilege database access, trigger behavior, and test-clone validation
Business Expert	Business-domain recommendations when product or operational meaning was unresolved
Finance Controller	Pricing, tax, freight, currency, rounding, zero-versus-unknown behavior, and finance-policy decisions for the proof of concept
Deployment Engineer	Local hosting, ports, launch workflow, runtime configuration, packaging, and operational readiness for the local proof of concept

The definitions do more than assign labels. They establish boundaries. For example, the API Engineer is instructed not to infer business rules from table names alone; the Product Designer owns the user outcome but not implementation architecture; the Finance Controller is the policy source for financial behavior; the Database Engineer verifies schema and persistence behavior but does not define product meaning; and the Deployment Engineer is limited to the local proof-of-concept runtime rather than inventing external deployment environments.

These boundaries reduced a common multi-agent failure mode: several capable agents independently making incompatible decisions in overlapping areas.



4.2 The Orchestrator

The Orchestrator is conceptually distinct from the eight specialists. Its primary purpose is not domain implementation. It maintains continuity across the system.

Across the audit record, the Orchestrator repeatedly performed functions analogous to an engineering lead, delivery coordinator, and verification owner:

- assigned and sequenced work;
- maintained backlog state and dependencies;
- handed questions to the appropriate specialist;
- recognized when an unresolved decision blocked further work;
- preserved audit history;
- reconciled documents and implementation;
- reran focused and full validation;
- prevented silent scope expansion;
- escalated consequential decisions to the human owner;
- coordinated corrections after specialist findings.

The audit contains 241 meaningful action/intervention rows. The Orchestrator appears in 90 of them, more than any other actor. This does not imply that the Orchestrator performed the majority of implementation. It indicates that coordination itself became a large and visible part of the engineering workload.

A useful conceptual representation of the final organization is:

Human Project Owner

↓ direction, authority, policy, risk acceptance

AI Orchestrator

↓ decomposition, coordination, validation, handoff, memory

Eight Specialist Agents

↓ product, architecture, frontend, API, database, business, finance, deployment

This structure is one of the central outcomes of the experiment.

5. Persistent Organizational Memory

AI conversations are transient compared with the institutional memory of a human engineering team. A developer may remember why a decision was made three weeks earlier. A team may remember an incident that caused a convention to be introduced. An agent should not be assumed to retain that history unless the project externalizes it.

The project therefore developed a set of persistent artifacts that served as organizational memory.

5.1 Backlog

`backlog.csv` became the shared work queue. It records feature ID, priority, status, target user, user story, rationale, frontend scope, API scope, acceptance criteria, dependencies, risks, confidence, owner agent, and next action.

The completed backlog contains 44 items:

Status	Count
Completed	38
Proposed	2
Deferred	2
Conditional	1
In Progress	1
Total	**44**

The six non-completed items are important. They should not be treated automatically as failures. Several were deliberately retained as deferred, proposed, or conditional because the local proof of concept did not justify silently converting unresolved production concerns into completed behavior. Scope control is itself an engineering result.

Ownership of the backlog shows meaningful specialization:

Owner	Items	Completed
Frontend Engineer	12	12
API Engineer	11	11

Owner	Items	Completed
Product Designer	6	3
Database Engineer	5	4
Orchestrator	5	4
Project Architect	3	2
Finance Controller	1	1
Deployment Engineer	1	1

5.2 Audit history

`docs/agent-audit.md` records who acted, what was done, and the result or evidence. It functions as a development flight recorder.

The audit contains 241 meaningful action rows. Role participation appears as follows; because a single row can name more than one participant, these counts are not additive percentages:

Actor or role	Audit appearances
Orchestrator	90
Human	48
API Engineer	41
Database Engineer	33
Product Designer	26
Project Architect	20
Frontend Engineer	12
Finance Controller	9
Deployment Engineer	6
Business Expert	3
Security Review agent	1

The audit is particularly valuable because it does not record only successes. It captures blocked decisions, failed assumptions, safety gates, corrected implementations, unverified conditions, deferred scope, and validation results. That makes the repository suitable for post-project analysis rather than merely demonstration.

5.3 Decision records, API contracts, data notes, and runbooks

The repository also preserves product decisions, architecture decisions, API contracts, database mapping and permission notes, financial-field analysis, and a development/test safety runbook. These artifacts externalize reasoning that would otherwise be buried in chat context.

The runbook is especially important because it turns repeated operational knowledge into a reusable procedure. This is conventional engineering practice, but in an agentic workflow it also reduces the amount of knowledge that must be reconstructed by later agents.

6. From Schema to Product

The first substantive challenge was not code generation. It was deciding what product should exist.

AdventureWorks contains many possible application surfaces: product browsing, sales operations, customer management, inventory, purchasing, reporting, and more. The Product Designer initially inferred a coherent internal sales-order workflow rather than attempting to expose the database as a collection of CRUD pages.

That decision led to a product sequence centered on:

- browsing and searching products;
- viewing product details and eligible offers;
- building a transient order draft;
- selecting a customer;
- choosing billing and shipping addresses;
- choosing a shipping method;
- reviewing calculated amounts;
- submitting an order;
- confirming persistence;
- looking up the resulting order.

This distinction matters. A database browser with forms would have been much easier to generate, but it would not have demonstrated domain reconstruction. The resulting system instead attempted to identify a plausible user goal and organize several database concepts around that goal.

The Product Designer also recorded uncertainty. The internal sales workflow was treated as a provisional assumption until implementation evidence and human direction made the target clearer. This behavior is preferable to disguising inference as requirement.

7. Architecture Derived from Verified Constraints

The Project Architect established a browser-to-API-to-database boundary rather than allowing direct browser access to SQL Server. Database credentials remained server-side. Input validation and authorization were treated as server responsibilities. SQL result sets were bounded. Sort/filter fields were allowlisted. Database identities were restricted by least privilege.

The architecture was influenced by actual database behavior rather than a generic template. AdventureWorks order persistence involves computed values, triggers, and related tables. As a result, order submission was treated as a transactional operation with database-owned effects rather than as a naive pair of independent inserts.

The system ultimately used:

- React 19 and Vite for the client;

- ASP.NET Core 10 for the API;
- SQL Server AdventureWorks2022 test clone for persistence;
- server-side `Microsoft.Data.SqlClient` access;
- DPAPI-protected local credential files;
- request-level Windows Negotiate authentication for the local proof of concept;
- least-privilege reader and writer identities with explicitly verified permissions;
- automated API and client tests;
- PowerShell launch and validation scripts.

A notable architectural characteristic is that potentially consequential behavior was progressively unlocked. Read-only functionality came first. Write behavior was not enabled merely because the schema contained writable tables. The team first established an isolated clone, verified trigger behavior, designed permissions, defined financial policy, and obtained explicit human authorization.

8. Development Chronology

The audit reveals a staged progression rather than a single generation event.

8.1 Phase 1 - Product and architecture discovery

On October 26, 2026, project roles and initial product scope were established. Product and architecture work defined the first usable slice and separated user-facing requirements from implementation responsibilities.

8.2 Phase 2 - Safe database access and isolated test environment

On October 27, the project established an isolated `AdventureWorks2022_Test` clone and a restricted read adapter. The API was deliberately configured to fail closed if the approved credential, database target, or permissions were absent.

This phase also exposed the importance of verification. The Orchestrator corrected assumptions about the protected credential file after inspection showed that it contained a password rather than a full connection string. A permission-query issue was also corrected to account for SQL Server 2022 behavior.

8.3 Phase 3 - Product completion and submission blockers

As the read-only product and customer flows matured, the project encountered decisions that could not be inferred safely from schema: pricing semantics, tax, freight, offer eligibility, order status, idempotency, and authorization boundaries.

Rather than forcing those decisions into implementation, the agents created explicit handoffs and blockers. This produced a less dramatic development narrative than “the AI just kept coding,” but a more credible engineering process.

8.4 Phase 4 - Backlog triage, validation, and operational hardening

The project worked through incomplete backlog items, added focused tests, strengthened body-size handling, improved immutable validation state, and introduced a Deployment Engineer to own local runtime concerns. Launch scripts were made explicit about API and client ports and were verified not to terminate unrelated listeners.

8.5 Phase 5 - Authentication and least-privilege order reading

A Security Review agent identified a flaw in the local Development authentication model: the API process identity was effectively being treated as the internal user, rather than authenticating the caller.

The issue was escalated. The human owner selected the intended local proof-of-concept audience rule. The Project Architect recommended actual Windows Negotiate authentication. The Deployment Engineer assessed feasibility. The API Engineer implemented the change. The Orchestrator independently verified same-origin authenticated UI and API behavior.

This is a useful example of cross-agent defect handling. The finding moved through security, human policy, architecture, deployment, implementation, and verification rather than being patched locally without system-level review.

8.6 Phase 6 - Controlled order-write enablement

The Database Engineer verified a clone-only writer identity and tested order-header/detail insertion inside an explicit transaction. Trigger-generated values and `TransactionHistory` effects were verified, and rollback was confirmed to leave header, detail, history, and error-log counts unchanged.

The human owner explicitly authorized only the isolated-clone work and prohibited automatic privilege expansion if the restricted path failed.

Only after that bounded database behavior, product decisions, financial rules, and authentication work were in place was the API order-submission route enabled.

9. Case Study: When Agents Stopped Instead of Guessing

One of the more important behaviors in the project was refusal to treat every unknown as a coding problem.

Several questions could not be answered from AdventureWorks alone:

- What price source should the proof of concept present?
- Should zero mean a known zero price or unavailable data?
- What tax rule should be demonstrated?
- What freight value should be used?
- What should happen if an offer or price changes between review and submission?
- What order status should a newly persisted order receive?
- How durable must idempotency be?

The agent organization separated these questions by authority. Finance questions went to the Finance Controller. Product semantics went to the Product Designer or Business Expert. Security and identity decisions were escalated rather than inferred. Human approval was requested for database grants and consequential writes.

For the local proof of concept, the resulting authorized finance behavior included a 7.5% demo tax and $25.00 flat freight, while preserving the distinction between demonstration policy and production authority.

This separation is significant because hallucination in software engineering is not limited to invented APIs or column names. An equally serious failure is invented policy: code that is syntactically correct and internally consistent but implements a business rule that no authorized person ever chose.

10. Case Study: Agent-to-Agent Error Detection

The experiment produced several examples in which one part of the agent organization identified or corrected another part.

A particularly clear instance occurred after the Project Owner approved an exact seven-column SELECT grant for order lookup on the test clone. The Database Engineer applied the approved column-level grants and then replayed the API's own permission-inspection SQL.

The grants themselves were correct. The API's permission guard was not.

The existing logic incorrectly interpreted column-level permissions. Under the actual SQL Server permission shape, the guard would report all seven expected permissions as missing and would also infer unexpected table-level permissions. The Database Engineer identified the mismatch and verified a corrected query shape. The API implementation was subsequently repaired.

This episode demonstrates an important advantage of role specialization when combined with shared evidence: the Database Engineer was not merely a code-writing persona. It brought a different verification perspective to an API implementation and exposed an error that ordinary unit-level reasoning could have missed.

The lesson is not that agents automatically review one another correctly. The system had to provide a reason for the Database Engineer to inspect the real permission behavior, preserve the result, and hand the finding back to implementation. The value came from the combination of specialization, verification, and orchestration.

11. Case Study: Security Finding and Architectural Correction

The Security Review finding on Development authentication is another instructive example.

The original local behavior relied on process identity in a way that did not actually authenticate the browser caller. For a single-user local proof of concept this might have appeared harmless, but it violated the system's own intended authorization boundary.

The response involved multiple roles:

1. The Security Review agent identified and described the issue.
2. The human owner selected the desired local rule: any successfully Windows-authenticated local user could act as `InternalStaff` for this proof of concept.
3. The Project Architect recommended request-level Windows Negotiate authentication with no process-identity fallback.
4. The Deployment Engineer evaluated the local browser/Kestrel path.
5. The API Engineer implemented same-origin authentication behavior.
6. The Orchestrator validated anonymous and authenticated behavior at runtime.

The final runtime check verified that anonymous UI access returned 401, authenticated UI and JavaScript assets returned 200, anonymous health remained available, authenticated order lookup failed closed when SQL credentials were intentionally unavailable, and unknown API routes returned 404.

The larger point is procedural. A security finding was not treated as a local code edit. It was translated through product authority, architecture, runtime feasibility, implementation, and independent verification.

12. Case Study: Safe Database Writes

Database writes represented the highest-risk transition in the experiment.

AdventureWorks was preserved as a reference source by default. The project created and verified an isolated test clone for write behavior. The Database Engineer inspected triggers, called modules, ownership, permissions, and execution context before proposing a restricted writer.

The human owner then authorized a narrow clone-only test: a collision-checked `AdventureWorksTestWriter`, default database fixed to `AdventureWorks2022_Test`, no mapping to the original database, DPAPI-protected password storage outside the repository, minimum required INSERT rights, and limited SELECT rights for reference and post-trigger verification.

Inside an explicit transaction, the Database Engineer verified:

- generated order identifiers and sales-order number;
- calculated line total, subtotal, and total due;
- revision behavior;
- transaction-history effects;
- rollback leaving tracked row counts unchanged.

Only later was a distinct proof-of-concept API write path enabled for committed demo orders.

This staged approach demonstrates a practical governance principle for agentic development: consequential capabilities should be unlocked through progressively verified boundaries rather than granted at the beginning of the experiment.

13. The Final Application

The final application is a functioning local website rather than a scaffold.

Its primary demonstrated workflow is:

1. Search the product catalog.
2. View a product.
3. Add a priced line to a transient order draft.
4. Set quantity and select an eligible offer.
5. Search for and select a customer.
6. Select billing and shipping addresses.
7. Select a shipping method.
8. Review server-derived order amounts.
9. Submit the order.
10. Receive a persisted sales-order reference.
11. Look up the stored order and its lines.

The README documents this workflow using `Mountain-200` and the customer `A Bike Store` as demonstration examples.

The final recorded live end-to-end validation used the local API at `http://localhost:18080`, with approved reader and writer credentials against the isolated clone. The browser workflow created order

SO75128 for customer 1, address 832, ship method 1, and product 782 with quantity 12 using the eligible "Volume Discount 11 to 14" offer.

The persisted header and detail matched the application confirmation. Recorded values included:

Field	Verified value
SubTotal	26,989.0824
TaxAmt	2,024.18
Freight	25.00
TotalDue	29,038.2624

The order lookup subsequently found the stored order and line. The API log recorded no errors during that verification. The order was intentionally left in the test clone as evidence of the completed demo.

14. Quantitative Results

14.1 Source and documentation size

Excluding common generated directories such as `node_modules`, `bin`, `obj`, and build output, the repository contains approximately:

Artifact	Lines	Files
C#	9,719	64
JavaScript	1,330	14
JSX	1,949	9
CSS	365	3
PowerShell	673	3
Markdown documentation	2,246	28

Lines of code are not a quality measure, but they establish that the result is materially larger than a trivial generated sample.

14.2 Automated validation

Static inspection of the repository finds 211 xUnit `[Fact]`/`[Theory]` declarations and 48 directly identifiable client `test`/`it` declarations. The runtime audit is more authoritative for final executed counts because test frameworks can expand cases dynamically.

The final recorded order-workflow validation reports:

- **287 API tests passed**;
- **8 opt-in SQL/integration tests skipped** in that validation context;
- **53 client tests passed**;
- client lint passed;
- production client build passed.

The value of these numbers is not simply volume. Testing was used as a coordination mechanism. Focused tests validated individual slices, and the Orchestrator repeatedly reran broader validation after changes to ensure that local progress did not silently break other parts of the system.

14.3 Backlog completion

Of 44 tracked backlog items, 38 reached `Completed`, or 86.4 percent. The remaining six retained explicit non-completed states because some questions were intentionally left outside the local proof-of-concept scope or required future production authority.

This is an important distinction between project completion and backlog exhaustion. The completed application met the bounded demonstration objective without pretending that every production concern had been solved.

15. Human Participation

The audit contains 48 rows involving the Human role. A review of those entries shows that human participation was concentrated in areas different from routine implementation.

The main categories were:

15.1 Direction

The human established the experiment, selected the overall objective, decided when to continue through incomplete work, and defined the desired outcome.

15.2 Authority

The human authorized actions that the agents were not entitled to approve for themselves, especially database grants, write behavior, and changes to security assumptions.

15.3 Product and business policy

When multiple plausible behaviors existed and the database could not establish intent, the human selected the rule or approved a specialist recommendation.

15.4 Scope control

The human accepted a local proof-of-concept boundary rather than requiring production-grade treatment of every future concern.

15.5 Risk acceptance

The human determined which limitations could remain explicit and which defects required correction before the project could proceed.

This suggests a useful interpretation of human-in-the-loop agentic development: the human did not vanish from software engineering. The human role moved upward in the decision hierarchy.

Implementation, analysis, testing, documentation, and coordination were heavily delegated. Intent, authority, consequential policy, and final risk ownership remained human responsibilities.

16. What Worked Well

Several patterns were consistently effective.

16.1 Specialization with explicit boundaries

Specialist agents were most useful when their authority was specific. The Database Engineer verified database facts rather than deciding product policy. The Finance Controller decided proof-of-concept financial rules rather than editing SQL. The Deployment Engineer owned local runtime behavior rather than system architecture as a whole.

16.2 Persistent artifacts

The backlog, audit, design documents, API contracts, data notes, and runbook reduced dependence on conversational memory. They also made the final project auditable.

16.3 Fail-closed behavior

The project repeatedly preferred unavailable functionality over silently unsafe behavior. Business routes stayed unavailable when credentials or permissions were absent. Order submission remained disabled while important gates were unresolved. Authentication was changed rather than relying on process identity.

16.4 Verification against the real system

Many important findings occurred only because agents checked actual runtime or database behavior. The permission-guard defect is the clearest example. A plausible implementation was wrong until compared with SQL Server's real permission model.

16.5 Narrow progression

The team generally advanced in slices: discover, design, implement, run focused tests, run broader validation, record the result, and then move to the next dependency. This created a workable rhythm for multi-agent development.

17. What Did Not Work Automatically

The experiment also exposed limitations.

17.1 The database could not supply business intent

Schema and data can suggest a domain, but they do not authorize business policy. Tax, freight, financial authority, availability, order status meaning, and production identity could not be safely derived from column names or sample values.

17.2 Individual agent competence did not guarantee system coherence

Specialists could produce individually reasonable work that still required coordination. Contracts, permissions, authentication, and deployment behavior crossed role boundaries.

17.3 Plausible code still required empirical verification

The column-permission guard looked reasonable until tested against the real SQL Server permission shape. Security behavior also required real request-level validation rather than static confidence.

17.4 The agents required bounded authority

Without explicit restrictions, an agent could theoretically solve a blocker by broadening privileges, changing scope, inventing policy, or bypassing safety. The project repeatedly prevented those shortcuts through role definitions and human approval gates.

17.5 Production readiness was not demonstrated

The completed system is a local proof of concept. Authentication hardening for a deployed environment, durable idempotency, production secret management, production database migration/backup strategy, external hosting, high availability, observability, compliance, and production financial authority were not established by this experiment.

18. Orchestration as a First-Class Engineering Concern

The strongest qualitative finding is that orchestration became as important as code generation.

A serious application requires decisions to remain compatible over time. Completed work must be recognized as complete. Blocked work must remain blocked. A new finding must reach the agent responsible for correcting it. Tests must be rerun after changes. Policy must not be silently converted into implementation. Human approvals must be recorded and scoped.

These are coordination problems.

The Orchestrator's 90 audit appearances provide a rough indication of how frequently that coordination work surfaced. It maintained the shared state within which specialist agents could operate.

This suggests that the future value of agentic development may depend less on adding more independent coding agents and more on improving the systems that govern them: task state, authority, evidence, memory, handoff, validation, and escalation.

19. A Model of Agentic Software Engineering

Based on this experiment, a practical agentic engineering system can be described as five interacting layers.

Layer 1 - Human intent and authority

A human defines the objective, owns consequential decisions, approves risk, and determines acceptable scope.

Layer 2 - Orchestration

An orchestrator decomposes work, routes questions, tracks dependencies, preserves state, and verifies progression.

Layer 3 - Specialized engineering roles

Agents apply domain-specific reasoning within explicit boundaries.

Layer 4 - Persistent engineering memory

Backlogs, audit records, contracts, decisions, runbooks, and source control preserve state outside transient conversations.

Layer 5 - Automated and empirical verification

Tests, builds, runtime checks, permission inspections, database transactions, and end-to-end workflows determine whether the system actually behaves as intended.

Removing any one of these layers would have weakened the experiment. Code generation without verification would have left plausible errors undiscovered. Specialists without orchestration would have lacked continuity. Orchestration without persistent artifacts would have depended too heavily on conversational state. Agents without bounded authority could have resolved blockers by making unauthorized decisions. Human governance without delegation would have reduced the experiment to ordinary manual development with AI autocomplete.

20. Implications for Senior Software Engineers

This experiment points toward a change in the work of experienced engineers rather than a simple elimination of that work.

A senior engineer using an agent organization may spend proportionally less time typing implementation code and more time on:

- decomposing ambiguous goals;
- designing the agent organization itself;
- defining authority and safety boundaries;
- creating persistent decision structures;
- reviewing cross-domain assumptions;
- deciding which evidence is sufficient;
- approving consequential actions;
- identifying when a model is confidently solving the wrong problem;
- translating business policy into verifiable engineering constraints;
- validating the integrated system rather than individual snippets.

These are recognizably software-engineering skills. Agentic development changes their leverage.

In this project, the ability to distinguish a database fact from a business decision was more important than the ability to generate another repository class. The ability to design a least-privilege progression was more important than rapidly enabling writes. The ability to preserve an audit trail made it possible to analyze the process after completion.

The engineer increasingly becomes the designer and governor of a software-producing system.

21. Limitations

The results should be interpreted within several important limits.

First, AdventureWorks is a well-structured sample database. A poorly documented legacy database with misleading names, hidden external dependencies, inconsistent data, or undocumented stored procedures would present a harder reconstruction problem.

Second, this was a single project conducted by one human owner. The experiment does not compare agentic development statistically against a conventional human team or a single general-purpose coding agent.

Third, the final system is a local proof of concept. It demonstrates a real transactional workflow, but not production deployment, production security, high availability, regulated-data compliance, performance at scale, or operational support over time.

Fourth, audit appearances are a process indicator, not a time-and-motion measurement. The experiment did not capture exact human minutes versus agent minutes, token consumption, or comparable labor cost for a conventional team.

Fifth, the agent organization evolved during the project. That evolution is itself a finding, but it means the experiment did not begin with a fixed controlled protocol.

Finally, the project was directed by an experienced software engineer. The effectiveness of this workflow may depend heavily on the human's ability to recognize architectural, security, data, and policy boundaries. A novice using the same agents might accept unsafe or incorrect recommendations that an experienced engineer would challenge.

22. Conclusions

The completed experiment demonstrates that a coordinated GitHub Copilot agent organization can do substantially more than generate isolated code snippets.

Starting principally from AdventureWorks2022, the agents inferred a useful internal sales-order workflow, established an architecture, implemented a React and ASP.NET Core application, created safe read access, defined and tested a controlled write path, built a local deployment workflow, generated extensive automated tests, documented decisions, and executed a verified end-to-end order transaction.

The final repository contains 44 tracked backlog items, 241 meaningful audit records, thousands of lines of application code and documentation, 287 passing API tests in the final recorded validation, 53 passing client tests, and a persisted test-clone order successfully retrieved through the application.

But the most important result is organizational.

Specialist agents alone were not enough. The project became effective when specialization was combined with orchestration, persistent organizational memory, explicit authority, empirical verification, and human governance.

The agents were capable of substantial autonomous engineering work, but they were also capable of plausible mistakes. The system succeeded not because mistakes disappeared, but because the process made it possible to expose, route, correct, and record them.

The human engineer likewise remained essential, but in a changed role. Much implementation and coordination could be delegated. Intent, authority, product meaning, risk acceptance, and final accountability could not.

The experiment therefore supports a more useful framing than "AI replaces programmers" or "AI is only autocomplete." A better description is:

> ***Agentic software engineering reorganizes the development process so that a human engineer can govern a software-producing organization of specialized AI agents, persistent project memory, explicit decision boundaries, and automated verification.***

For this project, that model was sufficient to move from a database to a functioning application.

Appendix A. Repository Evidence Map

The public repository is intended to make the experiment inspectable rather than requiring readers to accept the paper's conclusions on trust alone.

Evidence	Repository location
Agent definitions	.github/agents/*.agent.md
Shared backlog	backlog.csv
Agent and human audit trail	docs/agent-audit.md
Authentication architecture	docs/architecture/authentication.md
Product decisions	docs/product/
API contracts and submission design	docs/api/
Database mappings and permission evidence	docs/data/
Development/test safety runbook	docs/runbooks/development-and-test-safety.md
ASP.NET Core API	src/server/AdventureWorks.Api/
React client	src/client/
API tests	tests/AdventureWorks.Api.Tests/
Local launch workflow	scripts/run-local.ps1
Full validation workflow	scripts/validate.ps1
Project overview and demo instructions	README.md

Appendix B. Reproducibility and Safety Notes

The repository should not be interpreted as authorization to run write operations against an arbitrary AdventureWorks database. The experiment deliberately used an isolated `AdventureWorks2022_Test` clone and separately restricted local reader and writer identities.

Public distribution should exclude secrets and machine-specific credentials. The project already stores local SQL passwords outside the repository in current-user DPAPI-protected files. A public-release review should additionally verify that no local artifacts, logs, generated files, connection strings, or credentials were accidentally committed.

Readers wishing to reproduce the application should begin with the repository README and the development/test safety runbook. The final local demonstration assumes the isolated clone and permission boundaries described by the project documentation.

Appendix C. Interpreting the Metrics

The numerical results in this paper are intended to make the experiment inspectable, not to claim universal productivity gains.

Backlog counts come from `backlog.csv`. Audit counts are based on meaningful table rows in `docs/agent-audit.md`. Source-line counts exclude common generated directories. Test execution counts are taken from the final recorded project validation in the audit. Static test-declaration counts are provided only as repository context and may differ from runtime-expanded test counts.

The most important evidence remains behavioral: the application was exercised through the browser, submitted a real order through the API into the isolated SQL Server clone, returned a confirmation, and then retrieved the persisted order through the application's order-lookup workflow.